Writeup for Plane Or Exchange crypto

> Nhan_laptop

Tổng quan

Ta có một protocol được sử dụng trao đổi khóa thông qua Diffie-Hellman dựa trên lý thuyết nút.

- Ta có cả 2 public key của Alice và Bob, tuy nhiên ta không thể tính được shared key từ 2 public key này.

Phân tích

1. Biểu diễn nút - Grid Diagram

Mỗi "point" là một cặp hoán vị `(x, o)` có cùng kích thước `n`. Mỗi phần tử đại diện cho tọa độ cột `n × n`:

- `x[i]` = cột đặt dấu "X" trên hàng `i`
- `o[i]` = cột đặt dấu "O" trên hàng `i`

Hàm `validate(point)` kiểm tra:

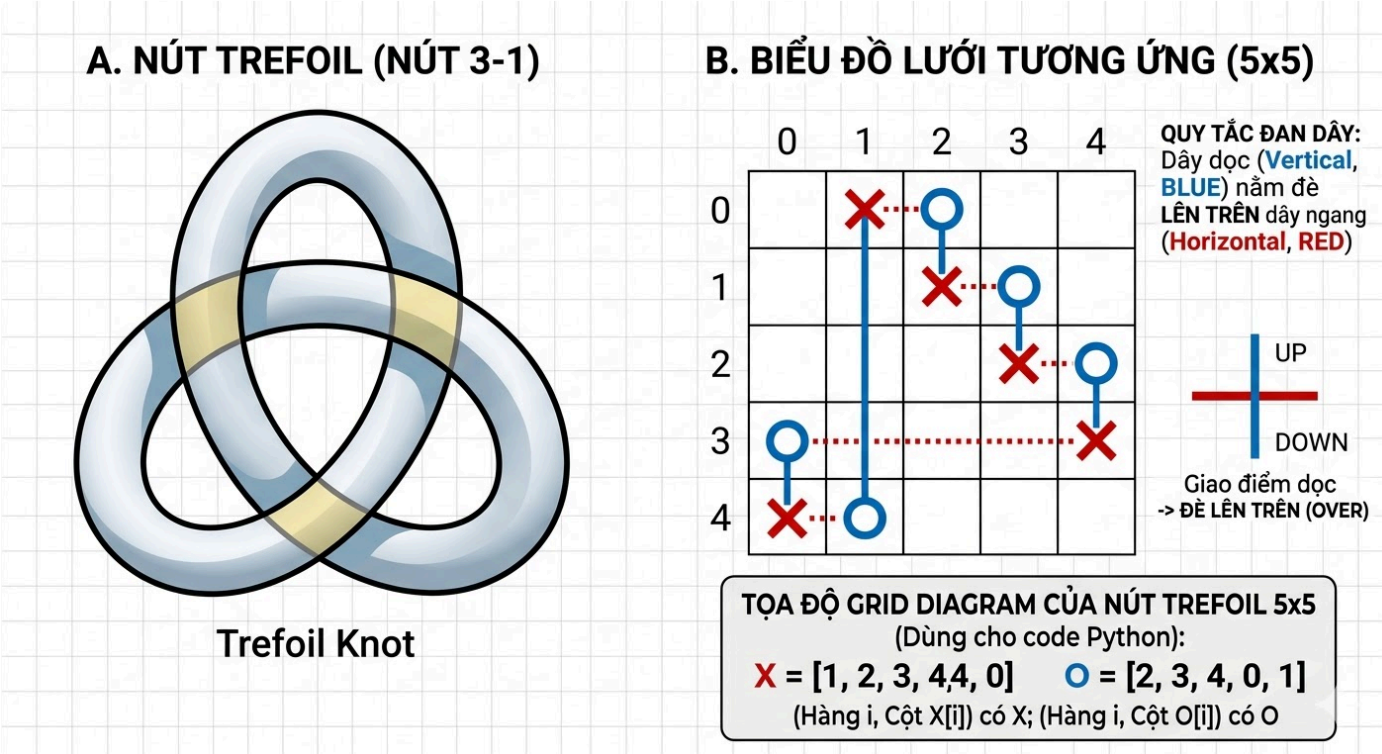
1. `len(x) == len(o)` — cùng kích thước.
2. `Permutation(compose(x, inverse(o))).cycles == 1` — hoán vị `x ∘ o⁻¹` là cyclic permutation duy nhất (đảm bảo đây là **một** nút, không phải link nhiều thành phần).
3. `sum(x[i] == o[i] ...) == 0` — không có hàng nào có X và O cùng cột.

2. Tính đa thức Alexander

```
def calculate(point):
    mat = sp.Matrix([[t**-x for x in y] for y in mine(point)])
    return mat.det(method='bareiss') * (1-t)**(1-len(point[0]))
```

Hàm `mine(point)` gọi `sweep()` để xây dựng **winding number matrix**:

```
def sweep(ap):
    l = len(ap)
    current_row = [0] * l
    matrix = []
    for pair in ap:
        c1, c2 = sorted(pair)
        diff = pair[1] - pair[0]
        if diff > 0:
            s = 1
        elif diff < 0:
            s = -1
        else:
            s = 0
        for c in range(c1, c2):
            current_row[c] += s
        matrix.append(list(current_row))
    return matrix

def mine(point):
    x, o = point
    return sweep([*zip(x, o)])
```



- `sweep()` duyệt qua từng cặp `(x[i], o[i])`, cập nhật `current_row` để tính số lần "đi qua" mỗi cột.
- Với mỗi cặp `(x[i], o[i])`, ta vẽ một đường ngang từ cột `x[i]` đến `o[i]`.
- Kết quả là ma trận Seifert (winding number matrix) `S[i][j]`.

Sau đó:

- Dựng ma trận `M[i][j] = t^(-S[i][j])`
- Tính `det(M) * (1-t)^(1-n)`

Kết quả sau khi normalize chính là **đa thức Alexander** $\Delta_K(t)$ — một **bất biến nút** (knot invariant) nổi tiếng.

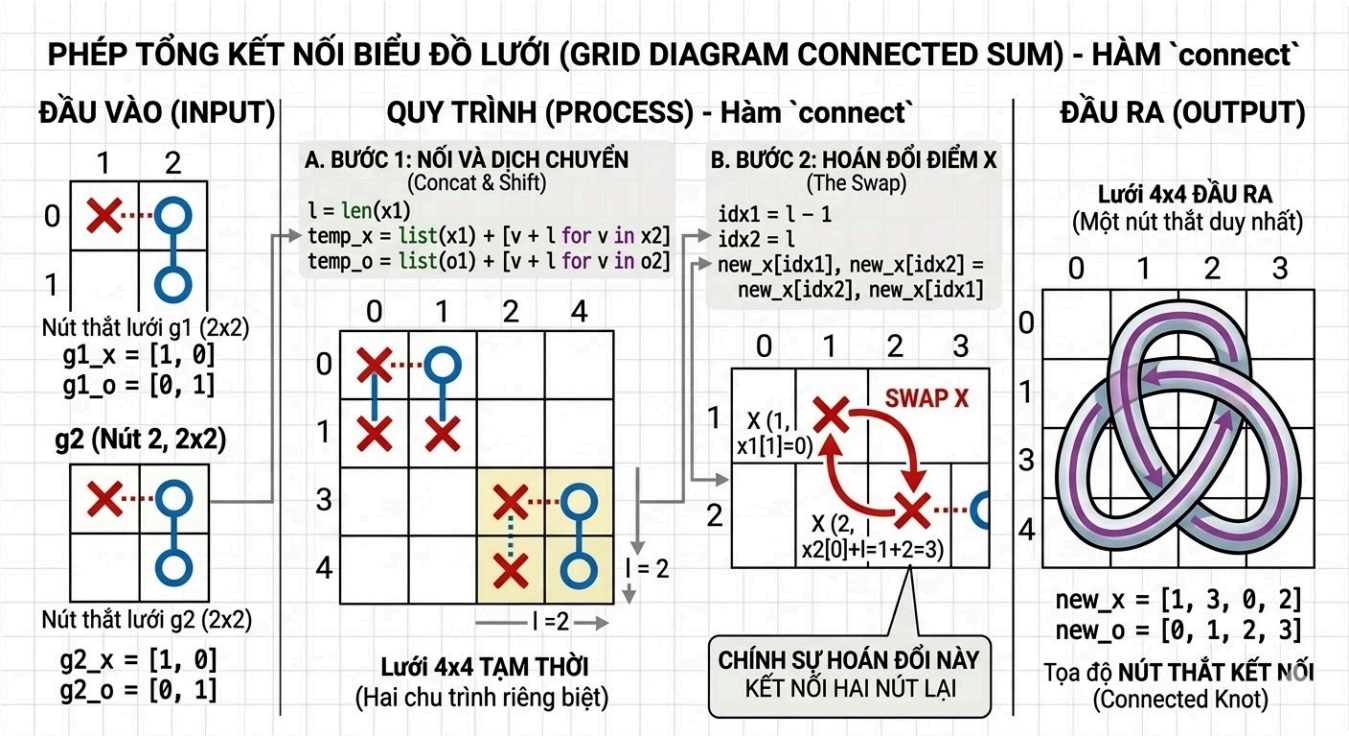
3. Tổng liên thông

```
def connect(g1, g2):
    x1, o1 = g1
    x2, o2 = g2
    l = len(x1)
    new_x = list(x1) + [v + l for v in x2]
    new_o = list(o1) + [v + l for v in o2]
    idx1 = l - 1
    idx2 = l
    new_x[idx1], new_x[idx2] = new_x[idx2], new_x[idx1]
    return (new_x, new_o)
```

Hàm này thực hiện **connected sum** (tổng liên thông) $K_1 \# K_2$ của hai nút:

1. Nối hai grid diagram lại bằng cách ghép lưới (dịch `g2` sang phải `l` đơn vị).

2. Hoán đổi phần tử x cuối cùng của `g1` với phần tử x đầu tiên của `g2` — tạo "cầu nối" giữa hai nút.



4. Scramble - Các biến đổi Reidemeister

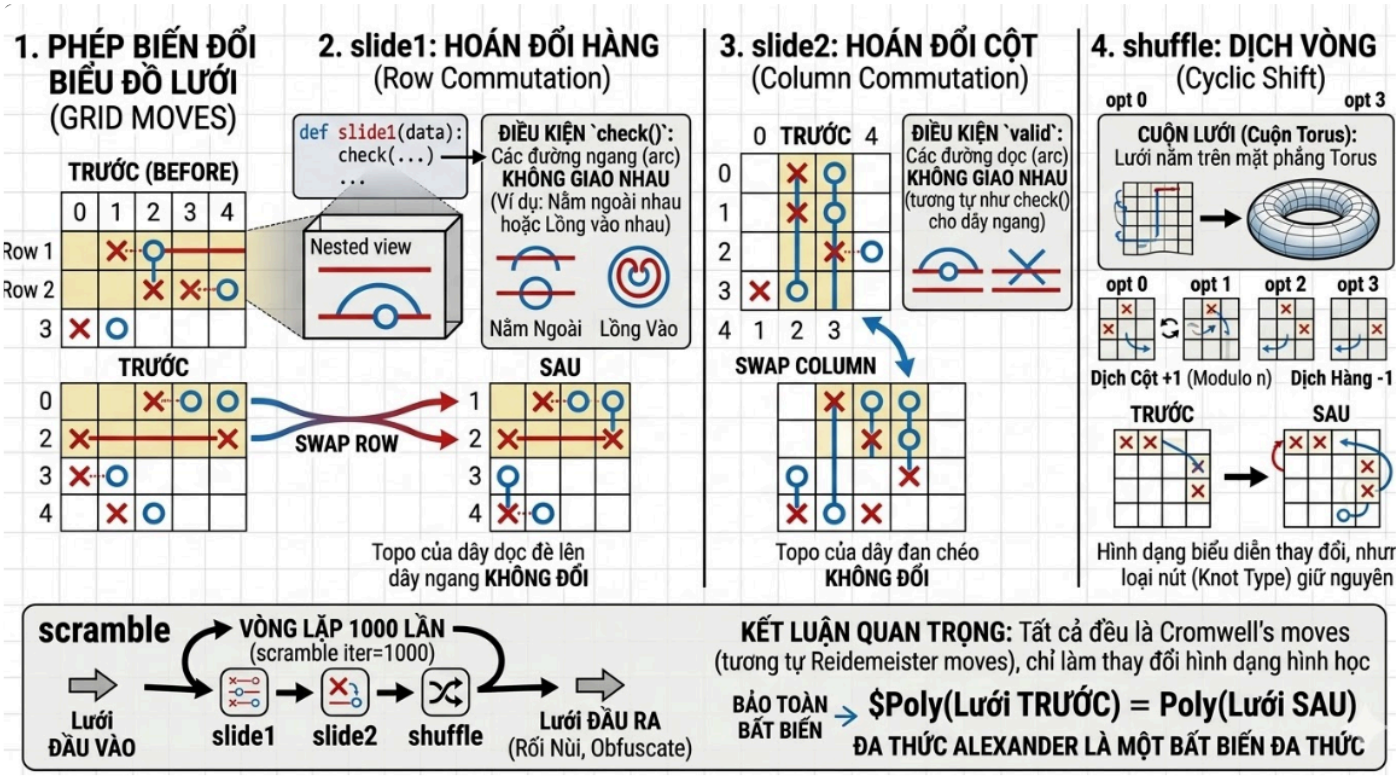
```
def slide1(data):
    def check(x1, o1, x2, o2):
        low1, high1 = min(x1, o1), max(x1, o1)
        low2, high2 = min(x2, o2), max(x2, o2)
        if high1 < low2 or high2 < low1:
            return True
        if (low1 > low2 and high1 < high2) or (low2 > low1 and high2 < high1):
            return True
        return False
    x, o = data
    idx = randbelow(len(x)-1)
    if check(x[idx], o[idx], x[idx+1], o[idx+1]):
        x[idx], x[idx+1] = x[idx+1], x[idx]
        o[idx], o[idx+1] = o[idx+1], o[idx]
    return (x, o)

def slide2(data):
    x_cols, o_cols = data
    n = len(x_cols)
    j = randbelow(n - 1)
    x_pos_j = x_cols.index(j)
    o_pos_j = o_cols.index(j)
    x_pos_j1 = x_cols.index(j + 1)
    o_pos_j1 = o_cols.index(j + 1)
    low_j, high_j = min(x_pos_j, o_pos_j), max(x_pos_j, o_pos_j)
    low_j1, high_j1 = min(x_pos_j1, o_pos_j1), max(x_pos_j1, o_pos_j1)
    valid = False
    if high_j < low_j1 or high_j1 < low_j:
        valid = True
    elif (low_j > low_j1 and high_j < high_j1) or (low_j1 > low_j and high_j1 < high_j):
        valid = True
    if valid:
        new_x = list(x_cols)
        new_o = list(o_cols)
        new_x[x_pos_j] = j + 1
        new_x[x_pos_j1] = j
        new_o[o_pos_j] = j + 1
        new_o[o_pos_j1] = j
        return (new_x, new_o)
    return data

def shuffle(data):
    x, o = data
    n = len(x)
    opt = randbelow(4)
    if opt==0:
        x = [(x+1)%n for x in x]
        o = [(o+1)%n for o in o]
    if opt==1:
        x = [(x+n-1)%n for x in x]
        o = [(o+n-1)%n for o in o]
    if opt==2:
        x = x[-1:] + x[:-1]
        o = o[-1:] + o[:-1]
    if opt==3:
        x = x[1:] + x[:1]
        o = o[1:] + o[:1]
    return (x, o)

def scramble(data, iter):
    new = [[*data[0]], [*data[1]]]
    for _ in range(iter):
        f = choice([
            slide1,
            slide2,
            shuffle
        ])
        new = f(new)
```

```
    return new
```

Hàm `scramble(data, iter)` áp dụng ngẫu nhiên 1000 lần một trong ba phép biến đổi:

- **`slide1`** : Hoán đổi hai hàng liền kề nếu các đường ngang (arc) của chúng không giao nhau.
- **`slide2`** : Hoán đổi hai cột liền kề (trị số `j` và `j+1`) nếu các arc tương ứng không giao nhau.
- **`shuffle`** : Dịch vòng (cyclic shift) tất cả các giá trị cột ±1, hoặc dịch vòng thứ tự hàng ±1.

Tất cả các phép biến đổi này đều là Reidemeister moves trên grid diagram — chúng thay đổi hình dạng biểu diễn nhưng **không thay đổi kiểu nút** (knot type). Do đó, **đa thức Alexander được bảo toàn**.



5. Giao thức trao đổi khóa

Giao thức hoạt động như sau:

Bước	Thao tác
Setup	Chọn một nút công khai P (`public_info`)
Alice	Chọn private key = nút a. Public key: $A = \text{scramble}(P\#a)$
Bob	Chọn private key = nút b. Public key: $B = \text{scramble}(P\#b)$
Alice tính	$\text{shared} = \text{normalize}(\Delta_{a\#B})$ = $\text{normalize}(\Delta_{a\#\text{scramble}(P\#b)})$
Bob tính	$\text{shared} = \text{normalize}(\Delta_{b\#A})$ = $\text{normalize}(\Delta_{b\#\text{scramble}(P\#a)})$

Vì scramble không thay đổi kiểu nút:

- Alice tính: $\Delta_{a\#(P\#b)} = \Delta_a \cdot \Delta_P \cdot \Delta_b$
- Bob tính: $\Delta_{b\#(P\#a)} = \Delta_b \cdot \Delta_P \cdot \Delta_a$

3. Lỗ hổng

Đa thức Alexander thỏa mãn **tính chất nhân cho tổng liên thông**:

$$\boxed{\Delta_{K_1\#K_2}(t) = \Delta_{K_1}(t) \cdot \Delta_{K_2}(t)}$$

Đây là một kết quả cổ điển trong lý thuyết nút (xem: Lickorish, "An Introduction to Knot Theory").

Từ dữ liệu ta có thể tính:

Giá trị	Đa thức Alexander
`public_info` = P	Δ_P
`alice_pub` = $\text{scramble}(P\#a) \simeq P\#a$	$\Delta_P \cdot \Delta_a$
`bob_pub` = $\text{scramble}(P\#b) \simeq P\#b$	$\Delta_P \cdot \Delta_b$

Shared secret cần tính:

$$\text{shared_poly} = \text{normalize}(\Delta_{a\#(P\#b)}) = \text{normalize}(\Delta_a \cdot \Delta_P \cdot \Delta_b)$$

Ta tính:

$$\Delta_a \cdot \Delta_P \cdot \Delta_b = \frac{(\Delta_P \cdot \Delta_a) \cdot (\Delta_P \cdot \Delta_b)}{\Delta_P} = \frac{\Delta_A \cdot \Delta_B}{\Delta_P}$$

Vậy:

$$\boxed{\text{shared_poly} = \text{normalize}\left(\frac{\Delta(\text{alice_pub}) \cdot \Delta(\text{bob_pub})}{\Delta(\text{pub_info})}\right)}$$

Không cần biết private key của Alice hay Bob!

Vấn đề : Đa thức Alexander **mất quá nhiều thông tin** về cấu trúc nút. Nhiều nút khác nhau có cùng đa thức Alexander (ví dụ: unknot và Kinoshita-Terasaka knot đều có $\Delta(t) = 1$).

Quan trọng hơn, vì Δ là **multiplicative homomorphism** từ monoid các nút (dưới phép tổng liên thông) vào vành đa thức, ta có thể tính shared secret chỉ từ dữ liệu public mà không cần factorize nút.

Solve

Tính `calculate()` trực tiếp bằng sympy yêu cầu tính **định thức ma trận symbolic**:

$$M[i][j] = t^{-S[i][j]}$$

Với Alice ($n = 27$) và Bob ($n = 30$), đây là định thức ma trận symbolic 27×27 và 30×30 — rất chậm vì sympy phải thao tác trên đa thức Laurent.

Thay vì tính định thức symbolic, ta:

1. evaluate: Thay t bằng các giá trị nguyên cụ thể, tính định thức **số** (nhanh hơn rất nhiều).
2. interpolate: Từ đủ nhiều cặp $(t_i, \det(M(t_i)))$, khôi phục đa thức bằng nội suy Newton.

Chuẩn hóa ma trận

Ma trận gốc có phần tử $t^{-S[i][j]}$ (mũ âm). Ta nhân hàng i với t^{R_i} (với $R_i = \max_j S[i][j]$) để tất cả mũ trở thành **không âm**:

$$M'[i][j] = t^{R_i - S[i][j]}$$

Khi đó: $\det(M') = t^{\sum R_i} \cdot \det(M)$, nên $\det(M') $ là đa thức (không phải Laurent polynomial), và ta có thể nội suy.

Tính bậc

Bậc tối đa của $\det(M')$ bị chặn bởi:

$$D = \sum_{i=1}^{n} (R_i - \min_j S[i][j])$$

(Vì mỗi hàng đóng góp tối đa $R_i - \min_j S[i][j]$ vào bậc định thức theo Leibniz formula.)

Cần $D + 1$ điểm đánh giá để nội suy chính xác đa thức bậc $\leq D$.

Kết quả

Với mỗi $t_k \in \{2, 3, \ldots, D+2\}$ (tránh $t = 0, 1$ vì gây suy biến):

$$v_k = \det\left([t_k^{R_i - S[i][j]}]_{i,j}\right)$$

Đây là định thức ma trận **số nguyên**

Nội suy Newton

Dùng phương pháp sai phân chia (Newton's divided differences):

$$P(t) = c_0 + c_1(t - t_0) + c_2(t - t_0)(t - t_1) + \ldots$$

Với c_k tính bởi bảng sai phân chia. Dùng `Fraction` (số hữu tỉ chính xác) để tránh sai số.

Sau đó mở ngoặc bằng phương pháp Horner ngược (từ hệ số cao nhất):

Khôi phục đa thức Alexander

Ta có:

$$\text{calculate}(K) = \det(M) \cdot (1-t)^{1-n} = \frac{\det(M')}{t^{\sum R_i}} \cdot (1-t)^{1-n} = \frac{\det(M')}{t^{\sum R_i} \cdot (1-t)^{n-1}}$$

Nên cần chia đa thức $\det(M')$ cho $(1-t)^{n-1}$:

$$(1-t)^{n-1} = \sum_{k=0}^{n-1} \binom{n-1}{k} (-1)^k t^k$$

Thực hiện **phép chia đa thức** (polynomial long division). Chia hết (remainder = 0) vì Alexander polynomial luôn là đa thức.

Cuối cùng dịch lũy thừa (shift) đi $-\sum R_i$ và normalize (đảm bảo hệ số tự do dương, lũy thừa nhỏ nhất = 0).

Các đa thức

$$\Delta_P(t) = t^6 - 5t^5 + 13t^4 - 17t^3 + 13t^2 - 5t + 1$$

$$\Delta_A(t) = t^{14} - 11t^{13} + 61t^{12} - 217t^{11} + 548t^{10} - 1032t^9 + 1494t^8 - 1687t^7 + 1494t^6 - 1032t^5 + 548t^4 - 217t^3 + 61t^2 - 11t + 1$$

$$\Delta_B(t) = 2t^{14} - 19t^{13} + 84t^{12} - 226t^{11} + 405t^{10} - 523t^9 + 540t^8 - 527t^7 + 540t^6 - 523t^5 + 405t^4 - 226t^3 + 84t^2 - 19t + 2$$

(cả ba đều là đa thức đối xứng — palindromic — đúng như tính chất của đa thức Alexander.)

Ta tính shared secret:

$$\text{shared_poly} = \text{normalize}\left(\frac{\Delta_A \cdot \Delta_B}{\Delta_P}\right)$$

Thực hiện phép chia đa thức $(\Delta_A \cdot \Delta_B) \div \Delta_P$ bằng `sympy.Poly.div()` . Phần dư bằng 0 (xác nhận đúng).

$$\text{shared_poly} = 2t^{22} - 31t^{21} + 234t^{20} - 1136t^{19} + 3959t^{18} - 10514t^{17}$$

$$+22120t^{16} - 37997t^{15} + 54813t^{14} - 68477t^{13} + 76653t^{12} - 79253t^{11}$$

$$+76653t^{10} - 68477t^{9} + 54813t^{8} - 37997t^{7} + 22120t^{6} - 10514t^{5}$$

$$+3959t^{4} - 1136t^{3} + 234t^{2} - 31t + 2$$

Tinhs Shared secret

```
shared_secret = SHA256(str(shared_poly))
              = "4ce5bc3bb44ed4018cae38ffcda94bc08b85d79f0e71a012a74549c82ac91a82"
```

XOR ciphertext với keystream sinh từ shared secret:

```
flag = dice{plane_or_planar_my_w0rds_4r3_411_knotted_up}
```

References

- AnIntroductiontoKnotTheory1997-Lickorish.pdf